INGENIERÍA DE SOFTWARE CON IA · 2026

Desarrollo Agéntico y Harness Engineering

Contexto, guardarraíles, memoria persistente y metodología profesional para dirigir agentes de IA

Impacto Real de la IA en 2026

90%

Adopción diaria

Desarrolladores programando con IA

45%

Código generado

Nuevo software escrito por agentes

30%

Confianza ciega

Resultados válidos sin supervisión

> El mercado ya no paga por teclear sintaxis, sino por orquestar arquitectura y validar criterio.

01 · CAMBIO DE PARADIGMA

Vibe Coding vs. Ingeniería de Software

⚠ Vibe Coding

Intuición pura: Pedir código rápido sin diseño

Cero revisión: Aceptar cambios a ciegas

Deuda técnica: Sistemas frágiles e inmantenibles

Ingeniería Agéntica

Arquitectura clara: Reglas definidas antes de actuar

Control humano: Supervisión en puntos críticos

Sostenibilidad: Pruebas, trazabilidad y evolución

¿Cómo Aprende y Predice un LLM?

🐕 EJEMPLO COTIDIANO

La foto de tu perra con gorra

Si entrenas a una IA con miles de fotos de tu perra, no memoriza una sola imagen: aprende sus **patrones** (hocico, orejas, pelaje).
Al verla después con gorra, reconoce los rasgos clave y calcula la probabilidad de que siga siendo ella.

Del Patrón a los Tokens

Tokens: Piezas de palabras (como piezas de Lego) que lee la IA

Predicción: No "piensa", adivina el siguiente token más probable

Ventana de contexto: Su escritorio de trabajo limitado en memoria

Desarrollo Agéntico y Harness Engineering

Desarrollo Agéntico

Programar dirigiendo agentes autónomos con supervisión humana.

Ingeniería Agéntica

Diseñar reglas, flujos y pruebas para sistemas confiables.

Harness Engineering

Construir el entorno y frenos que controlan al agente.

Arnés, Guardarraíles y Context Engineering

Arnés (Harness) — El coche y el tablero

El motor solo (LLM) no puede viajar. El arnés es el sistema alrededor que le da herramientas para leer archivos, ejecutar comandos y trabajar por objetivos.

Guardarraíles (Guardrails) — Las barandas de la ruta

Límites y permisos para evitar accidentes: qué archivos no tocar, pedir aprobación antes de borrar y pasar tests antes de terminar.

Context Engineering — La mochila justa para la misión

Seleccionar, ordenar y actualizar la información exacta que el agente necesita en cada paso para no confundirse ni inventar datos.

03 · ECOSISTEMA 2026

Herramientas para el Desarrollo Agéntico



OPEN-SOURCE · AGNÓSTICO

OpenCode

Agente libre en terminal o editor con cualquier modelo.

ANTHROPIC · TERMINAL

Claude Code

Razonamiento profundo sobre proyectos complejos y MCP.

OPENAI · SANDBOXES

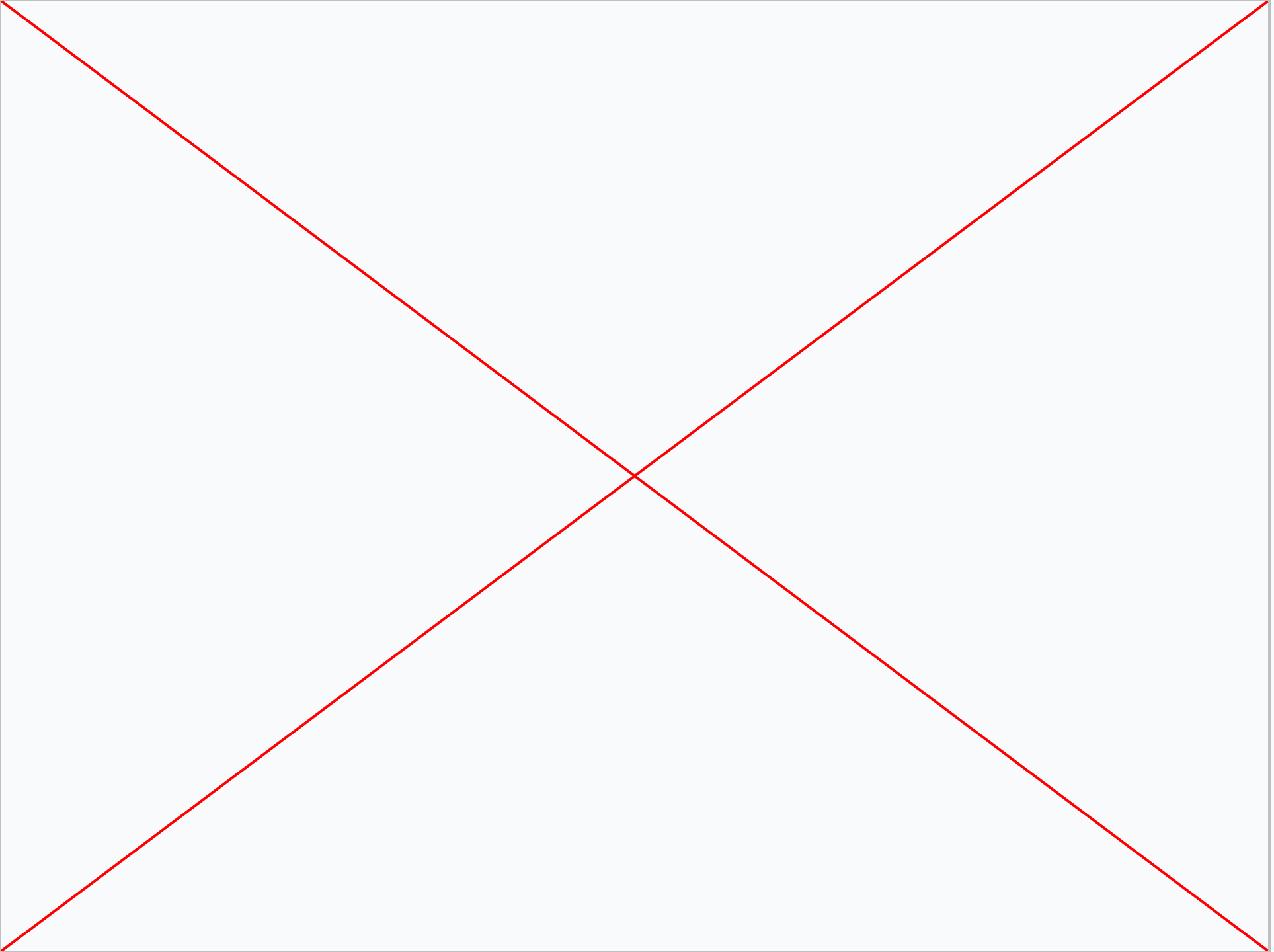
OpenAI Codex

Ejecución autónoma en paralelo dentro de entornos aislados.

GOOGLE · IDE AGÉNTICO

Google Antigravity

Orquestación entre editor, terminal y navegador con Skills.



opencode

Harness Avanzado: Comandos, Skills y MCP

/comandos

Comandos Custom

Atajos repetibles con prompts predefinidos.

@skills

Agent Skills

Capacidades modulares bajo demanda.

MCP Protocol

Model Context Protocol

Conexión estándar a APIs y datos.

Modo Plan vs. Modo Build

Modo Plan (Solo Lectura)

Propósito: Analizar, detectar riesgos y proponer pasos
Ideal para: Tareas grandes, refactors y arquitectura
Seguridad: No toca archivos hasta tu aprobación

Modo Build (Ejecución)

Propósito: Escribir código, ejecutar comandos y tests

Ideal para: Correcciones obvias y planes ya aprobados
Control: Auditable con /diff y reversible con /undo

Flujo de Trabajo y Puntos de Control Humano

FASE 1 · PREPARAR

Contexto y Reglas

Definir AGENTS.md y spec en Markdown.

FASE 2 · CONTROL

Modo Plan

Validar enfoque y casos límite antes de tocar código.

FASE 3 · EJECUTAR

Iteración Pequeña

Modo Build con cambios acotados y sesión limpia.

FASE 4 · CONTROL

Auditar y Persistir

Revisar /diff, tests y actualizar MEMORY.md.

Control obligatorio siempre: Arquitectura, permisos/seguridad, esquemas de datos, revisión de /diff y cierre de memoria.

SÍNTESIS · EL NUEVO PROGRAMADOR

La IA ejecuta el código. Tú gobiernas el sistema.

Fundamentos de ingeniería + Context Engineering + Guardarraíles deterministas